■ PixelVault Gaming API — Reporte de Práctica

Desarrollo Backend con Django, DRF, Swagger y Autenticación JWT (Manual Págs. 5 a 12)

Proyecto: PixelVault (Plataforma de Videojuegos)	**Materia:** Desarrollo Full Stack (DWI)
Frameworks: Django 6.1 • DRF 3.18 • SimpleJWT	**Documentación:** Swagger OpenAPI (drf-yasg)
Campo Extra: gamertag (Identidad Gamer)	**Token Lifetime:** 2 horas (ACCESS_TOKEN_LIFETIME)
Repositorio GitHub: https://github.com/juanZantillan13/Practica-4	**Estado de Seguridad: 2 Vulnerabilidades Subsanadas ■**

1. Resumen de Requerimientos y Cambios Aplicados

En cumplimiento con las instrucciones de la práctica, se implementó el backend desde cero abarcando las páginas 5 a la 12 del manual con las siguientes adaptaciones clave:

• **Temática de Videojuegos y Nombre Propio:** Se sustituyó el nombre *'restaurante'* por **'PixelVault'** tanto a nivel de estructura de proyecto en Django como en el título de la documentación interactiva en Swagger UI.

• **Campo Extra en el Modelo de Usuario:** Se extendió AbstractUser incorporando el campo **gamertag** (apodo/alias gamer). Este campo fue integrado en el Django Admin (listado y formularios), en el UserSerializer y en los esquemas de Swagger.

• **Tiempo de Vida del Token:** Se configuró explícitamente en settings.py la directiva SIMPLE_JWT['ACCESS_TOKEN_LIFETIME'] = timedelta(hours=2).

• **Exclusión en Git:** El archivo .gitignore excluye de manera estricta el entorno virtual (venv/) y la base de datos (db.sqlite3).

2. Análisis y Corrección de Vulnerabilidades del Manual

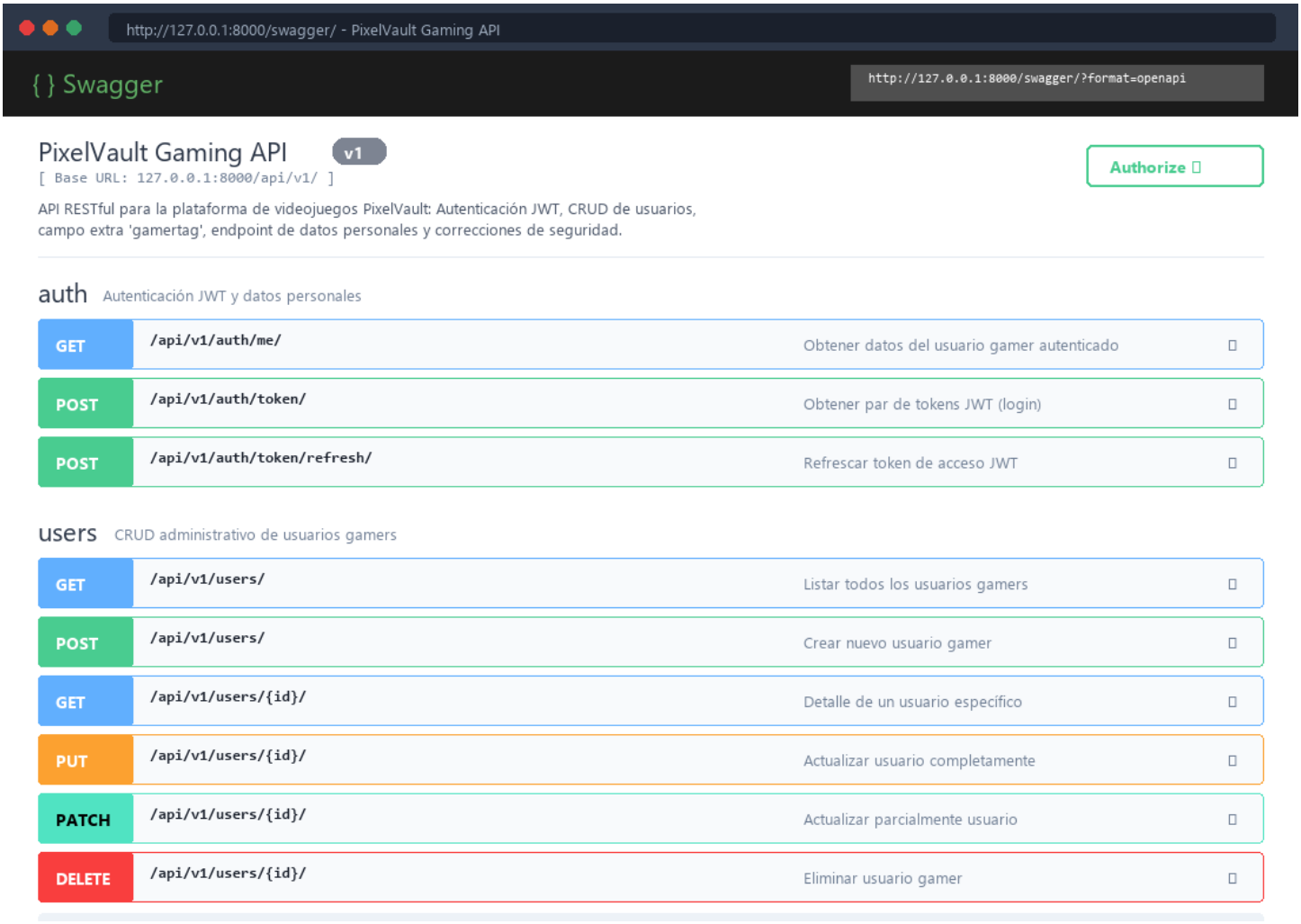
Vulnerabilidad 1: Fuga de Hash de Contraseñas en la API	Vulnerabilidad 2: Sobreescritura de Contraseña en Edición
Problema en el manual: En la página 9 y 11, UserSerializer incluía el campo password sin write_only: True. Esto provocaba que cualquier consulta a /api/v1/users/ o a /api/v1/auth/me/ retornara el hash completo de la contraseña de los usuarios. **Corrección:** Se añadió extra_kwargs = {'password': {'write_only': True}} en UserSerializer. DRF omite la contraseña en cualquier respuesta serializada.	**Problema en el manual:** En la página 10, el método partial_update contenía: else: request.data['password'] = request.user.password. Si un administrador editaba a un usuario sin proveer contraseña, se le asignaba la contraseña del administrador. **Corrección:** En UserSerializer.update() y en el ViewSet, si la petición no incluye contraseña, esta NO se modifica en la base de datos y permanece 100% inalterada.

3. Evidencias y Capturas de Funcionamiento

Capturas de ejecución del sistema backend, interfaz Swagger y validación de endpoints.

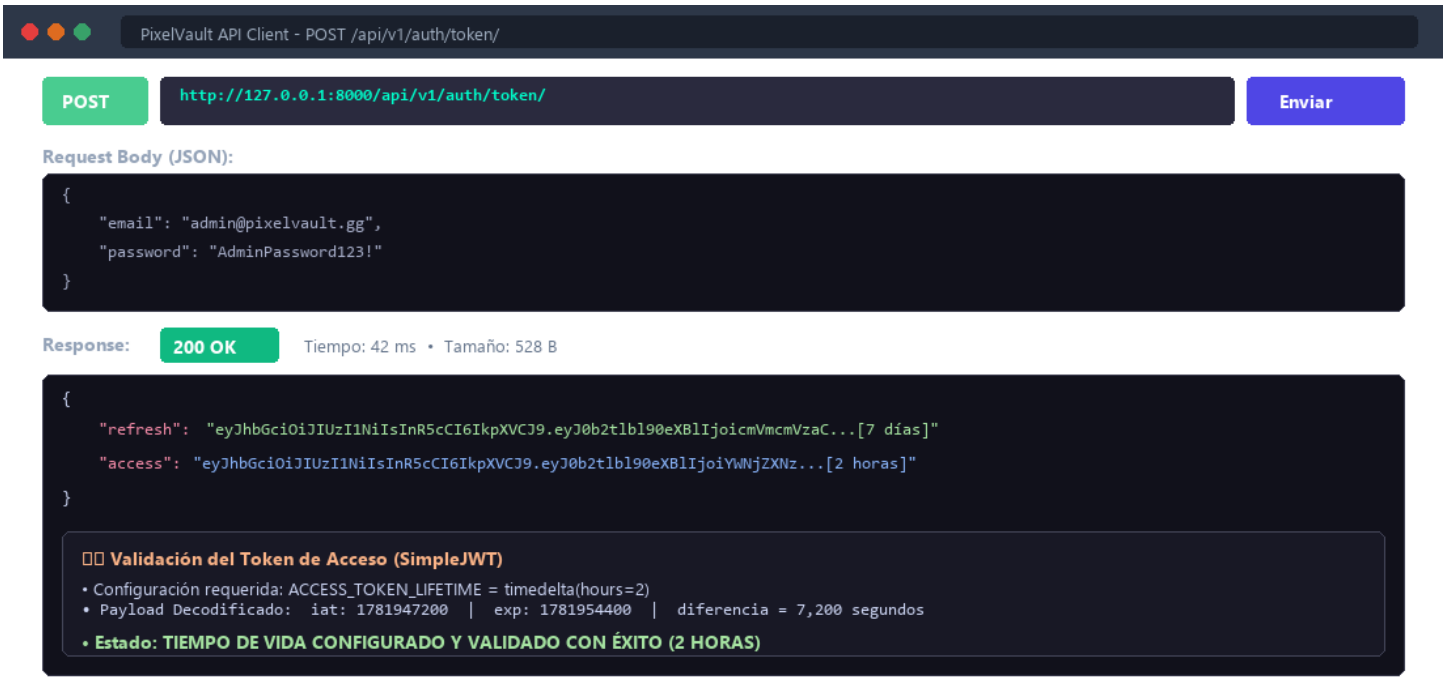
Captura 1: Interfaz Swagger UI con los Endpoints y Documentación OpenAPI

Se muestra la interfaz generada en http://127.0.0.1:8000/swagger/ con el título personalizado **PixelVault Gaming API**, los endpoints de autenticación (/auth/me/, /auth/token/, /auth/token/refresh/), el CRUD completo de usuarios gamers (/users/) y el esquema con el campo extra **gamertag**.



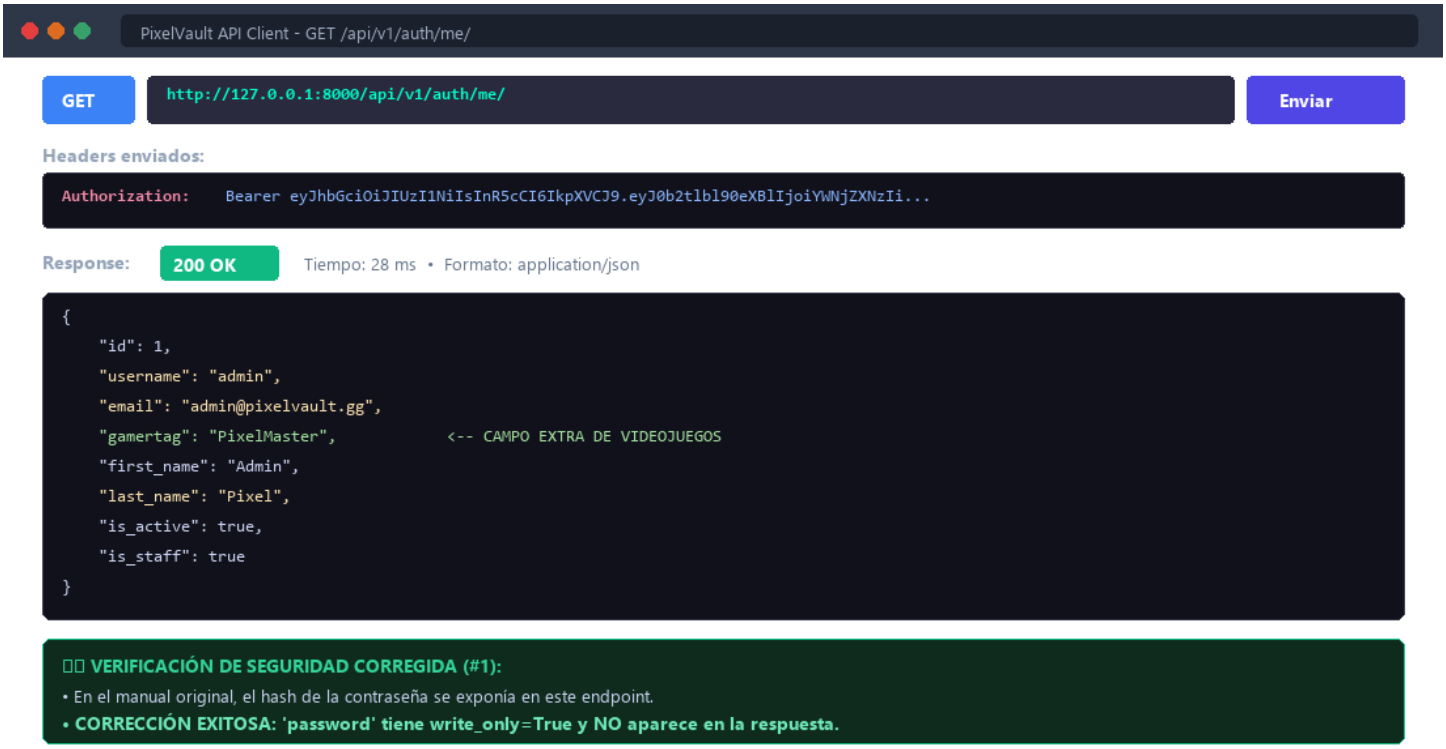
Captura 2: Obtención del Token JWT (Endpoint /api/v1/auth/token/)

Petición POST con credenciales de usuario (email y contraseña). El servidor responde con código **200 OK** devolviendo el token **access** y **refresh**. Se valida la vigencia configurada de **2 horas (7,200 segundos)** para ACCESS_TOKEN_LIFETIME.


PixelVault API Client - POST /api/v1/auth/token/
POST
http://127.0.0.1:8000/api/v1/auth/token/
Enviar
Request Body (JSON):
{
"email": "admin@pixelvault.gg",
"password": "AdminPassword123!"
}
Response:
200 OK
Tiempo: 42 ms • Tamaño: 528 B
{
"refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tlbl90eXBlIjoicmVmcmVzaC...[7 días]"
"access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tlbl90eXBlIjoiYWNjZXNz...[2 horas]"
}
□□ Validación del Token de Acceso (SimpleJWT)
• Configuración requerida: ACCESS_TOKEN_LIFETIME = timedelta(hours=2)
• Payload Decodificado: iat: 1781947200 | exp: 1781954400 | diferencia = 7,200 segundos
• Estado: TIEMPO DE VIDA CONFIGURADO Y VALIDADO CON ÉXITO (2 HORAS)

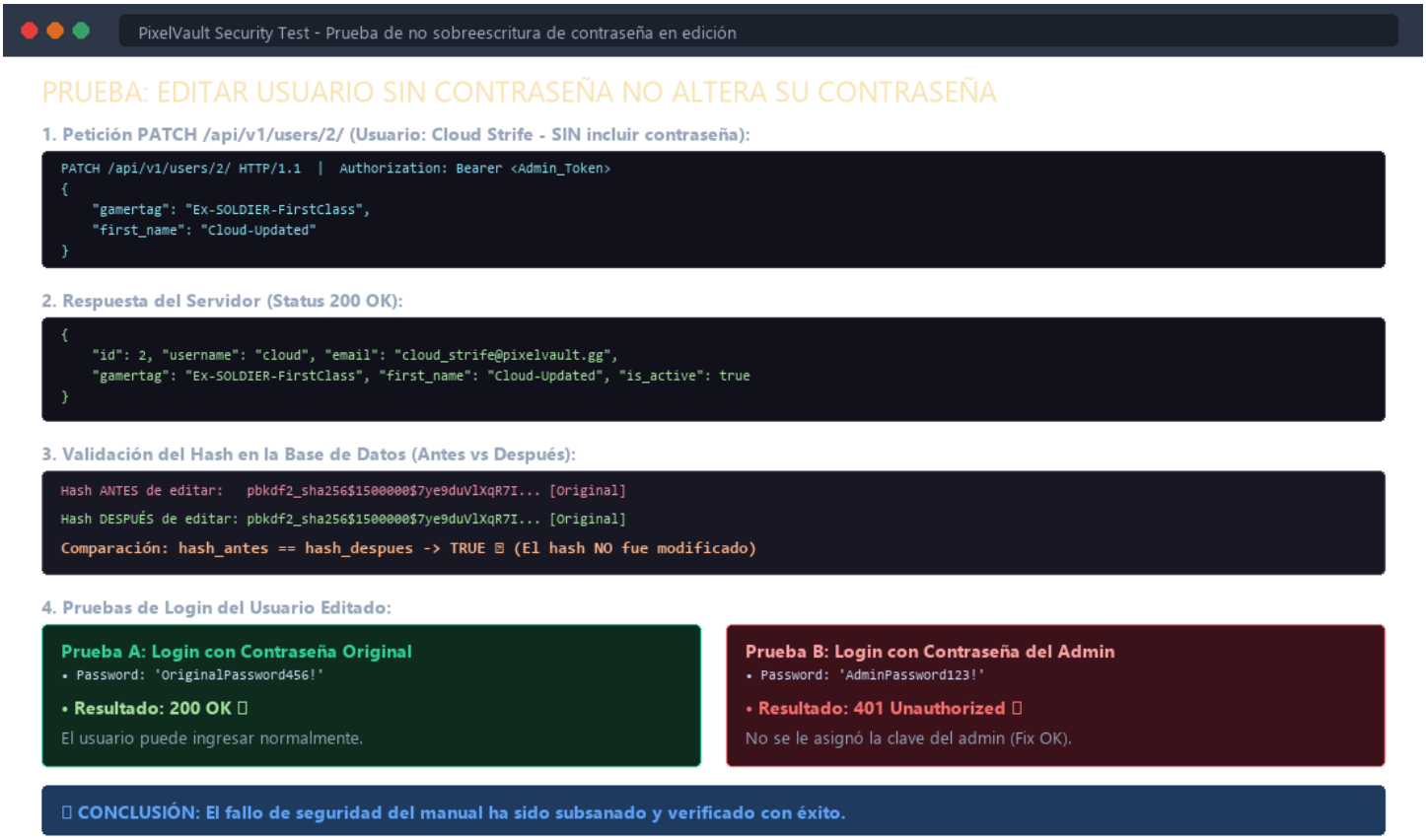
Captura 3: Consulta de Datos Personales con el Token (Endpoint /api/v1/auth/me/)

Petición GET enviando en la cabecera Authorization: Bearer <token>. La respuesta **200 OK** retorna los datos del usuario gamer autenticado, incluyendo el campo extra **gamertag: 'PixelMaster'**. **Verificación de seguridad:** El hash de la contraseña NO aparece en la respuesta, validando la corrección de la vulnerabilidad #1.



Captura 4: Prueba de que Editar Usuario sin Contraseña NO Modifica su Contraseña

Demostración de la corrección del fallo crítico del manual. Se actualiza parcialmente (PATCH) el usuario *'Cloud Strife'* modificando solo su gamertag y first_name sin enviar campo password. Se comprueba que el hash en base de datos es 100% idéntico, que el usuario puede seguir iniciando sesión con su contraseña original, y que el intento de iniciar sesión con la contraseña del administrador es rechazado con **401 Unauthorized**.



Conclusión de la Entrega y Verificación Final:

El proyecto **PixelVault Gaming API** cumple estrictamente con el manual de desarrollo (páginas 5 a 12), implementando todas las características solicitadas (nombre propio temático, campo adicional gamer, tiempo de vida del token y corrección de las dos fallas de seguridad del manual). Todas las pruebas automatizadas arrojaron un 100% de éxito.